

08-Defining-Functions

January 20, 2025

This notebook contains an excerpt from the [Whirlwind Tour of Python](#) by Jake VanderPlas; the content is available [on GitHub](#).

The text and code are released under the [CC0](#) license; see also the companion project, the [Python Data Science Handbook](#).

[< Control Flow](#) | [Contents](#) | [Errors and Exceptions >](#)

1 Defining and Using Functions

So far, our scripts have been simple, single-use code blocks. One way to organize our Python code and to make it more readable and reusable is to factor-out useful pieces into reusable *functions*. Here we'll cover two ways of creating functions: the `def` statement, useful for any type of function, and the `lambda` statement, useful for creating short anonymous functions.

1.1 Using Functions

Functions are groups of code that have a name, and can be called using parentheses. We've seen functions before. For example, `print` in Python 3 is a function:

```
[1]: print('abc')
```

```
abc
```

Here `print` is the function name, and `'abc'` is the function's *argument*.

In addition to arguments, there are *keyword arguments* that are specified by name. One available keyword argument for the `print()` function (in Python 3) is `sep`, which tells what character or characters should be used to separate multiple items:

```
[2]: print(1, 2, 3)
```

```
1 2 3
```

```
[3]: print(1, 2, 3, sep='--')
```

```
1--2--3
```

When non-keyword arguments are used together with keyword arguments, the keyword arguments must come at the end.

1.2 Defining Functions

Functions become even more useful when we begin to define our own, organizing functionality to be used in multiple places. In Python, functions are defined with the `def` statement. For example, we can encapsulate a version of our Fibonacci sequence code from the previous section as follows:

```
[4]: def fibonacci(N):  
    L = []  
    a, b = 0, 1  
    while len(L) < N:  
        a, b = b, a + b  
        L.append(a)  
    return L
```

Now we have a function named `fibonacci` which takes a single argument `N`, does something with this argument, and `returns` a value; in this case, a list of the first `N` Fibonacci numbers:

```
[5]: fibonacci(10)
```

```
[5]: [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

If you're familiar with strongly-typed languages like `C`, you'll immediately notice that there is no type information associated with the function inputs or outputs. Python functions can return any Python object, simple or compound, which means constructs that may be difficult in other languages are straightforward in Python.

For example, multiple return values are simply put in a tuple, which is indicated by commas:

```
[6]: def real_imag_conj(val):  
    return val.real, val.imag, val.conjugate()  
  
r, i, c = real_imag_conj(3 + 4j)  
print(r, i, c)
```

```
3.0 4.0 (3-4j)
```

1.3 Default Argument Values

Often when defining a function, there are certain values that we want the function to use *most* of the time, but we'd also like to give the user some flexibility. In this case, we can use *default values* for arguments. Consider the `fibonacci` function from before. What if we would like the user to be able to play with the starting values? We could do that as follows:

```
[7]: def fibonacci(N, a=0, b=1):  
    L = []  
    while len(L) < N:  
        a, b = b, a + b  
        L.append(a)  
    return L
```

With a single argument, the result of the function call is identical to before:

```
[8]: fibonacci(10)
```

```
[8]: [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

But now we can use the function to explore new things, such as the effect of new starting values:

```
[9]: fibonacci(10, 0, 2)
```

```
[9]: [2, 2, 4, 6, 10, 16, 26, 42, 68, 110]
```

The values can also be specified by name if desired, in which case the order of the named values does not matter:

```
[10]: fibonacci(10, b=3, a=1)
```

```
[10]: [3, 4, 7, 11, 18, 29, 47, 76, 123, 199]
```

1.4 *args and **kwargs: Flexible Arguments

Sometimes you might wish to write a function in which you don't initially know how many arguments the user will pass. In this case, you can use the special form `*args` and `**kwargs` to catch all arguments that are passed. Here is an example:

```
[11]: def catch_all(*args, **kwargs):  
      print("args =", args)  
      print("kwargs = ", kwargs)
```

```
[12]: catch_all(1, 2, 3, a=4, b=5)
```

```
args = (1, 2, 3)  
kwargs = {'a': 4, 'b': 5}
```

```
[13]: catch_all('a', keyword=2)
```

```
args = ('a',)  
kwargs = {'keyword': 2}
```

Here it is not the names `args` and `kwargs` that are important, but the `*` characters preceding them. `args` and `kwargs` are just the variable names often used by convention, short for “arguments” and “keyword arguments”. The operative difference is the asterisk characters: a single `*` before a variable means “expand this as a sequence”, while a double `**` before a variable means “expand this as a dictionary”. In fact, this syntax can be used not only with the function definition, but with the function call as well!

```
[14]: inputs = (1, 2, 3)  
      keywords = {'pi': 3.14}  
  
      catch_all(*inputs, **keywords)
```

```
args = (1, 2, 3)  
kwargs = {'pi': 3.14}
```

1.5 Anonymous (lambda) Functions

Earlier we quickly covered the most common way of defining functions, the `def` statement. You'll likely come across another way of defining short, one-off functions with the `lambda` statement. It looks something like this:

```
[15]: add = lambda x, y: x + y
      add(1, 2)
```

```
[15]: 3
```

This lambda function is roughly equivalent to

```
[16]: def add(x, y):
      return x + y
```

So why would you ever want to use such a thing? Primarily, it comes down to the fact that *everything is an object* in Python, even functions themselves! That means that functions can be passed as arguments to functions.

As an example of this, suppose we have some data stored in a list of dictionaries:

```
[17]: data = [{'first': 'Guido', 'last': 'Van Rossum', 'YOB': 1956},
              {'first': 'Grace', 'last': 'Hopper', 'YOB': 1906},
              {'first': 'Alan', 'last': 'Turing', 'YOB': 1912}]
```

Now suppose we want to sort this data. Python has a `sorted` function that does this:

```
[18]: sorted([2,4,3,5,1,6])
```

```
[18]: [1, 2, 3, 4, 5, 6]
```

But dictionaries are not orderable: we need a way to tell the function *how* to sort our data. We can do this by specifying the `key` function, a function which given an item returns the sorting key for that item:

```
[19]: # sort alphabetically by first name
      sorted(data, key=lambda item: item['first'])
```

```
[19]: [{'YOB': 1912, 'first': 'Alan', 'last': 'Turing'},
      {'YOB': 1906, 'first': 'Grace', 'last': 'Hopper'},
      {'YOB': 1956, 'first': 'Guido', 'last': 'Van Rossum'}]
```

```
[20]: # sort by year of birth
      sorted(data, key=lambda item: item['YOB'])
```

```
[20]: [{'YOB': 1906, 'first': 'Grace', 'last': 'Hopper'},
      {'YOB': 1912, 'first': 'Alan', 'last': 'Turing'},
      {'YOB': 1956, 'first': 'Guido', 'last': 'Van Rossum'}]
```

While these key functions could certainly be created by the normal, `def` syntax, the `lambda` syntax is convenient for such short one-off functions like these.

< [Control Flow](#) | [Contents](#) | [Errors and Exceptions](#) >